P2419R1
Clarify handling of encodings in localized formatting of chrono types

Published Proposal, 2021-09-19

Authors:

Victor Zverovich

Peter Brett

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Proposal

C++20 added formatting of chrono types with `std::format` but left unspecified what happens during localized formatting when the locale and literal encodings do not match ([LWG3565]).

Consider the following example:

```
std::locale::global(std::locale("Russian.1251"));
auto s = std::format("День недели: {}", std::chrono::Monday);
```

where `"День недели"` means `"Day of week"` in Russian.

(Note that `"{}"` should be replaced with `"{:L}"` if [P2372] is adopted but that's non-essential.)

If the literal encoding is UTF-8 and the "Russian.1251" locale exists we have a mismatch between encodings. As far as we can see the standard doesn't specify what happens in this case.

One possible and undesirable result (mojibake) is

```
"День недели: \xcf\xed"
```

where `"\xcf\xed"` is `"Пн"` (`"Mon"` in Russian) in CP1251 and is not valid UTF-8.

Another possible and desirable result is

```
"День недели: Пн"
```

where everything is in one encoding (UTF-8).

We propose clarifying the specification to prevent mojibake when possible by allowing implementation do transcoding or substituting the locale so that the result is in a consistent encoding.

This issue is not resolved by [LWG3547] / [P2372], the latter only reduces the scope of the problem to format strings with the `L` specifier only. The resolution proposed here is compatible with [P2372].

§ 2. Changes since R0

- Added more SG16 poll results.

§ 3. SG16 polls

SG16 Unicode reviewed [LWG3547] and there was a strong support for the direction of this paper. SG16 poll results:

Require implementations to make `std::chrono` substitutions with `std::format` as if transcoded to UTF-8 when the literal encoding E associated with the format string is UTF-8, for an implementation-defined set of locales.

SF	F	N	A	SA
1	6	2	0	0

Consensus: Consensus in favour.

Permit such substitutions when the encoding E is any Unicode encoding form.

SF	F	N	A	SA
0	7	2	0	0

Consensus: Consensus in favour.

Prohibit such substitutions otherwise.

SF	F	N	A	SA
1	3	3	1	1

Consensus: No consensus.

SA reason: Over-constrains implementations. May be sensible for implementations to perform all conversions uniformly.

Poll: Forward P2419R0 to LEWG as the recommended resolution of LWG 3565 and with a recommended ship vehicle of C++23.

SF	F	N	A	SA
4	2	1	0	0

Consensus: Strong consensus in favour.

§ 4. Implementation experience

The proposal has been implemented in the open-source {fmt} library ([FMT]) which includes chrono formatting facilities and tested on a variety of platforms.

§ 5. Wording

All wording is relative to the C++ working draft [N4892].

Update the value of the feature-testing macro `__cpp_lib_format` to the date of adoption in [version.syn]:

Change in [time.format]:

Each conversion specifier *conversion-spec* is replaced by appropriate characters as described in Table [tab:time.format.spec]; the formats specified in ISO 8601:2004 shall be used where so described. Some of the conversion specifiers depend on the locale that is passed to the formatting function if the latter takes one, or the global locale otherwise. If the string literal encoding is a Unicode encoding form and the locale is among an implementation-defined set of locales, each replacement that depends on the locale is performed as if the replacement character sequence is transcoded to the string literal encoding. If the formatted object does not contain the information the conversion specifier refers to, an exception of type `format_error` is thrown.

§ 6. Acknowledgement

Thanks Hubert Tong for bringing up this issue during the discussion of [P2093].

§ References

§ Informative References

[FMT]
Victor Zverovich; et al. The fmt library. URL: https://github.com/fmtlib/fmt

[LWG3547]
Victor Zverovich. Time formatters should not be locale sensitive by default. URL: https://cplusplus.github.io/LWG/issue3547

[LWG3565]

Victor Zverovich. Handling of encodings in localized formatting of chrono types is underspecified. URL: https://cplusplus.github.io/LWG/issue3565

[N4892]

Thomas Köppe. Working Draft, Standard for Programming Language C++. 18 June 2021. URL: https://wg21.link/n4892

[P2093]

Victor Zverovich. Formatted output. URL: https://wg21.link/p2093

[P2372]

Victor Zverovich; Corentin Jabot. Fixing locale handling in chrono formatters. URL: https://wg21.link/p2372